МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ
Кафедра «Программная инженерия»

Лабораторная работа № 5
по дисциплине «Математические основы баз данных»
по теме: «Транзакции, целостность данных и серверная логика»

Выполнил студент группы:
БВТ2505
Коновалова Варвара Вячеславовна

Проверил:
Быватов Семён Юрьевич

Москва 2026

Оглавление

Введение 3

Выполнение 4

Блок 4. Триггер аудита 7

Вывод 10

Введение

Цель и задачи работы:

Целью данной лабораторной работы является приобретение практических навыков использования транзакций, обеспечения целостности данных и реализации серверной бизнес-логики (функций на PL/pgSQL и триггеров) в СУБД PostgreSQL. Работа продолжает развитие базы данных «Онлайн-зоомагазин».

Задачи:

1. Продемонстрировать работу транзакции: выполнить связанные изменения между BEGIN и COMMIT, затем повторить сценарий с намеренной ошибкой и ROLLBACK.
2. Реализовать функцию на PL/pgSQL, атомарно оформляющую заказ (списание остатка и генерация исключения при нехватке товара).
3. Реализовать функцию, возвращающую сводный показатель с использованием переменных и запроса внутри функции.
4. Создать триггер аудита: при изменении цены товара старое и новое значение заносятся в таблицу журнала.
5. Создать триггер, поддерживающий денормализованное поле (пересчет среднего рейтинга товара при добавлении отзыва).
6. Обосновать, какую логику уместно держать в базе данных, а какую - в приложении.

Выполнение

Лабораторная работа выполняется на основе базы данных «Онлайн-зоомагазин», спроектированной и наполненной в предыдущих работах.

Для выполнения работы код разделен на логические блоки, которые выполняются последовательно.

Блок 1. Демонстрация работы транзакций (COMMIT и ROLLBACK)
В маркетплейсе оформление заказа требует согласованного добавления записи в таблицу orders, позиций в order_items и списания остатков из stock. Сначала выполняется откат транзакции с ошибкой (ROLLBACK), чтобы сбросить состояние прерванной транзакции.

```
-- 1.1. Отмена неудачной транзакции, чтобы снять ошибку 25P02
ROLLBACK;
```

```
-- 1.2. Транзакция с ошибкой
BEGIN;
INSERT INTO orders (user_id, status, total) VALUES (1, 'new', 0)
RETURNING id;
INSERT INTO order_items (order_id, product_id, quantity, unit_price)
VALUES (1202, 7, 1000, 4916.12);
UPDATE stock SET quantity = quantity - 1000 WHERE product_id = 7;
```

```
ERROR:  INSERT или UPDATE в таблице "order_items" нарушает
ограничение внешнего ключа "order_items_order_id_fkey"
Ключ (order_id)=(1202) отсутствует в таблице "orders".

ОШИБКА:  INSERT или UPDATE в таблице "order_items" нарушает
ограничение внешнего ключа "order_items_order_id_fkey"
SQL state: 23503
Detail: Ключ (order_id)=(1202) отсутствует в таблице "orders".
```

Здесь намеренно указан несуществующий `order_id` = 1202. СУБД выдаст ошибку нарушения внешнего ключа (foreign key constraint). Транзакция перейдет в прерванное состояние.

```
-- 1.3. Сбросить ошибку
ROLLBACK;
```

```
-- 2.1. Успешная транзакция (получение ID)
BEGIN;
INSERT INTO orders (user_id, status, total) VALUES (1, 'new', 0)
RETURNING id;
```

	id [PK] bigint
1	1209

```
-- 2.2. Успешная транзакция (завершение)
INSERT INTO order_items (order_id, product_id, quantity, unit_price)
VALUES (1209, 5, 2, 1366.08);
UPDATE stock SET quantity = quantity - 2 WHERE product_id = 5;
COMMIT;
```

```
COMMIT
```

Мы подставляем полученный `id` = 1209, добавляем позиции заказа и списываем товар со склада. Команда `COMMIT` успешно фиксирует все изменения в базе.

Блок 2. Функция на PL/pgSQL: Атомарное оформление заказа
Функция объединяет логику создания заказа и списания остатка в единую атомарную операцию. При нехватке товара возбуждается исключение (RAISE EXCEPTION).

```
-- 3. Создание функции place_order
CREATE OR REPLACE FUNCTION place_order(p_user BIGINT,
p_product BIGINT, p_qty INT)
RETURNS BIGINT AS $$
DECLARE
   v_order_id BIGINT;
   v_price NUMERIC(12,2);
BEGIN
   SELECT price INTO v_price FROM products WHERE id = p_product;
```

```
    IF (SELECT quantity FROM stock WHERE product_id = p_product) < p_qty THEN
        RAISE EXCEPTION 'Недостаточно товара на складе (id=%)', p_product;
    END IF;

    INSERT INTO orders (user_id, status, total)
    VALUES (p_user, 'new', v_price * p_qty)
    RETURNING id INTO v_order_id;

    INSERT INTO order_items (order_id, product_id, quantity, unit_price)
    VALUES (v_order_id, p_product, p_qty, v_price);

    UPDATE stock SET quantity = quantity - p_qty WHERE product_id = p_product;

    RETURN v_order_id;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE FUNCTION
```

Блок 3. Функция со сводным показателем

Функция get_brand_revenue вычисляет суммарную выручку выбранного бренда за указанный период времени, игнорируя отмененные заказы.

```
-- 4. Создание функции get_brand_revenue
CREATE OR REPLACE FUNCTION get_brand_revenue(p_brand_id BIGINT, p_start_date DATE, p_end_date DATE)
RETURNS NUMERIC(12,2) AS $$
DECLARE
    v_total_revenue NUMERIC(12,2);
BEGIN
    SELECT COALESCE(SUM(oi.quantity * oi.unit_price), 0) INTO v_total_revenue
    FROM order_items oi
    JOIN orders o ON o.id = oi.order_id
```

```
    JOIN products p ON p.id = oi.product_id
    WHERE p.brand_id = p_brand_id
      AND o.status <> 'cancelled'
      AND o.created_at::date BETWEEN p_start_date AND p_end_date;

    RETURN v_total_revenue;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE FUNCTION
```

Блок 4. Триггер аудита

Для логирования изменений цены создана таблица price_audit и триггер, который срабатывает после (AFTER) успешного обновления цены товара.

```
-- 5.1 Создание таблицы аудита
CREATE TABLE price_audit (
id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
product_id BIGINT NOT NULL,
old_price NUMERIC(12,2),
new_price NUMERIC(12,2),
changed_at TIMESTAMPTZ DEFAULT now()
);
```

```
CREATE TABLE
```

```
-- 5.2 Создание триггерной функции
CREATE OR REPLACE FUNCTION audit_price_change()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.price <> OLD.price THEN
        INSERT INTO price_audit (product_id, old_price, new_price)
        VALUES (NEW.id, OLD.price, NEW.price);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE FUNCTION
```

```
-- 5.3 Привязка триггера к таблице products
CREATE TRIGGER trg_audit_price
AFTER UPDATE OF price ON products
FOR EACH ROW EXECUTE FUNCTION audit_price_change();
```

```
CREATE TRIGGER
```

Блок 5. Триггер, поддерживающий денормализованное поле

Для ускорения выборки добавлено поле avg_rating к таблице products, которое автоматически пересчитывается при добавлении новых отзывов.

```
-- 6.1 Добавление столбца в таблицу
ALTER TABLE products ADD COLUMN avg_rating NUMERIC(3,2)
DEFAULT 0;
```

```
ALTER TABLE
```

```
-- 6.2 Создание триггерной функции
CREATE OR REPLACE FUNCTION refresh_product_rating()
RETURNS TRIGGER AS $$
BEGIN
   UPDATE products
   SET avg_rating = (
      SELECT COALESCE(ROUND(AVG(score), 2), 0)
      FROM reviews
      WHERE product_id = NEW.product_id
   )
   WHERE id = NEW.product_id;
   RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE FUNCTION
```

```
-- 6.3 Привязка триггера к таблице reviews
CREATE TRIGGER trg_refresh_rating
AFTER INSERT OR UPDATE ON reviews
FOR EACH ROW EXECUTE FUNCTION refresh_product_rating();
```

6. Логика в базе данных или в приложении
Размещение логики на стороне СУБД (триггеры, функции PL/pgSQL) имеет ряд преимуществ: правила применяются единообразно независимо от клиента, гарантируется строгая целостность данных на уровне хранилища, отсутствуют издержки на пересылку больших объемов данных по сети. СУБД отлично подходит для логики, неразрывно связанной с консистентностью данных (каскадные транзакции, пересчет денормализованных полей, аудит изменений). Однако прикладную бизнес-логику (отправка email-уведомлений, интеграция с внешними API, сложная алгоритмическая обработка) следует располагать в коде приложения. Серверный SQL-код сложнее масштабировать, тестировать и версионировать. Чрезмерное использование хранимых процедур повышает нагрузку на базу данных как единую точку отказа и усложняет миграцию на другие СУБД.

Вывод

В ходе выполнения лабораторной работы были изучены и применены на практике механизмы управления транзакциями в СУБД PostgreSQL. Реализована серверная бизнес-логика с помощью функций на языке PL/pgSQL для атомарного оформления заказов и расчета агрегированных показателей онлайн-зоомагазина. Созданы триггеры аудита и перерасчета денормализованных значений, что позволило обеспечить высокую консистентность данных. Также были проанализированы подходы к распределению бизнес-логики между сервером базы данных и приложением.